

دستیار جلسه

پیاده سازی فنی

سند ۲ از ۳ · مرجع مهندسی MVP فعلی · شرح پروژه · نقشه راه · CHANGELOG

این سند مرجع مهندسی سامانه‌ی در حال اجراست: اینکه داده چگونه از ورودی تا Jira جریان می‌یابد، هر مؤلفه چگونه ساخته شده، سطح API، رابط کاربری، مدیریت خطا، و مجموعه‌ی تست‌ها. برای انگیزه و چشم‌انداز محصول به [سند ۱](#) و برای کارهای آینده به [سند ۳](#) مراجعه کنید.

فهرست

- | | |
|-----------------------------------|---------------------|
| ۱. هدف سامانه | ۹. Endpoint های API |
| ۲. MVP در برابر نسخه کامل (YAGNI) | ۱۰. داده مصنوعی |
| ۳. جریان داده End-to-End | ۱۱. تنظیمات محیط |
| ۴. رونویسی صوت | ۱۲. ساختار پروژه |
| ۵. SQLite – ذخیره جلسات | ۱۳. اجرای سامانه |
| ۶. ChromaDB – Vector Store | ۱۴. تست‌ها |
| ۷. عامل‌های Pydantic AI | ۱۵. مدیریت خطا |
| ۸. رابط کاربری (Astro SSR) | |

۱. هدف سامانه

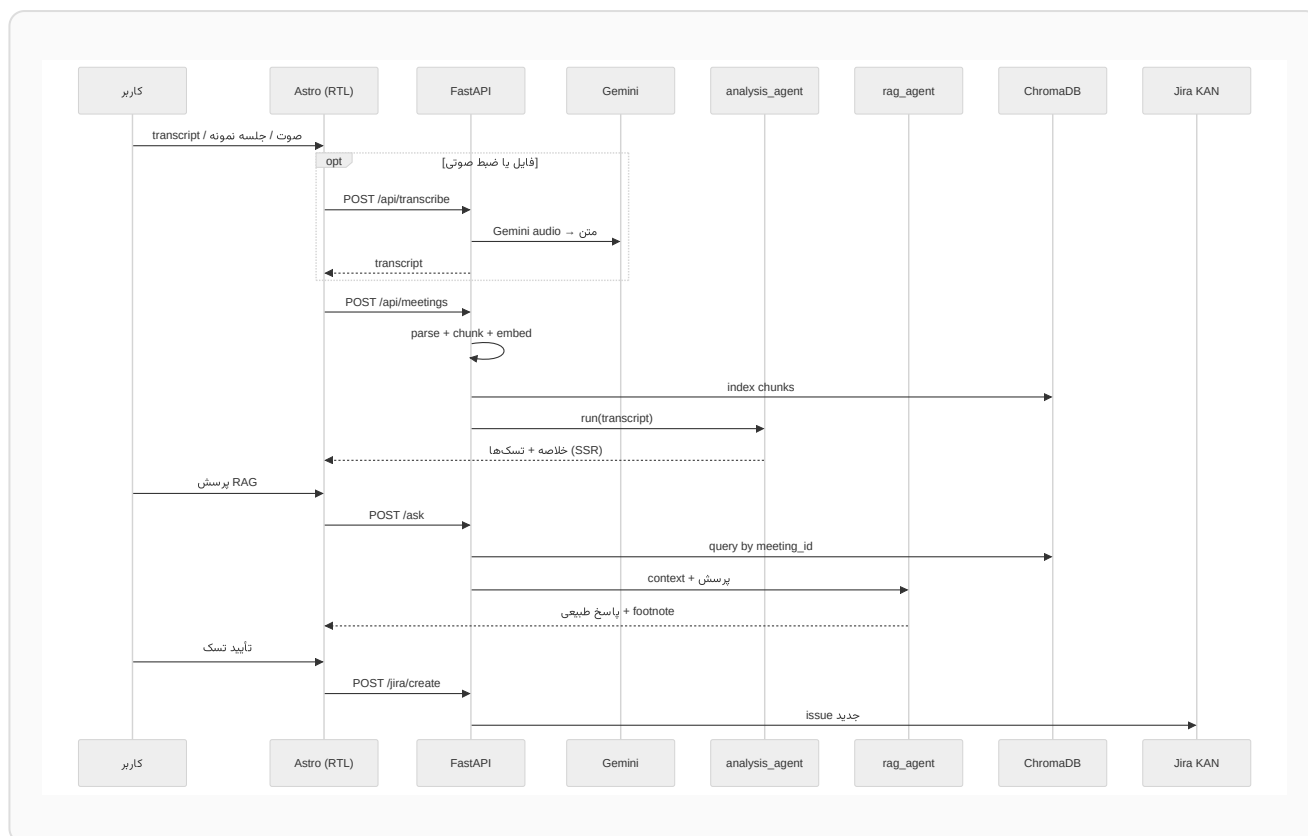
تبدیل transcript جلسات فارسی به خروجی عملیاتی: خلاصه، تسک، پرسش و پاسخ RAG، و [issue](#)های Jira – از طریق خط لوله‌ای که برای دموی زنده به اندازه‌ی کافی سریع است.

۲. MVP در برابر نسخه کامل (YAGNI)

سند مفهومی اولیه یک سامانه‌ی حداکثری را توصیف می‌کرد. MVP فقط آنچه را که برای اثبات ارزش لازم است نگه می‌دارد و به‌صراحت ثبت می‌کند که چه چیزی به تعویق افتاده – یک موضع منضبط «به آن نیاز نخواهی داشت».

قابلیت	نسخه کامل (سند مفهومی)	MVP فعلی
ورودی	ضبط زنده + STT جریانی	transcript (paste + فایل متنی) و صوت (آپلود/ضبط ← Gemini)
تحلیل	GPT-4 / مدل‌های متنوع	Gemini Flash + Pydantic AI
جستجو / Vector DB	ElasticSearch	(ChromaDB embedded (data/chroma/
ردیابی تسک	Jira + Trello + Asana ... +	فقط issue – (KAN) Jira ها انگلیسی
RAG	آرشیو کل سازمان	تک‌جلسه · چندجلسه برنامه‌ریزی‌شده (Sprint 2)
تقویم / Teams	یکپارچگی کامل	به تعویق افتاده (sprint بعد)
داشبورد / اعلان	گزارش هفتگی	داشبورد تسک cross-meeting امروز · گزارش‌های غنی‌تر بعداً

۳. جریان داده End-to-End



۴. رونویسی صوت (Gemini)

ورودی صوت یک مرحله‌ی جدا پیش از ingest است، تا paste متنی و بارگذاری جلسات نمونه بدون تغییر بمانند. همان GOOGLE_API_KEY و GEMINI_MODEL استفاده می‌شود — بدون سرویس speech-to-text جدا.

مرحله	جزئیات
UI	آپلود mp3, wav, m4a, ogg, webm, flac یا دکمه‌ی ضبط (MediaRecorder).
API	POST /api/transcribe — multipart file ← Gemini generateContent با صوت inline.
خروجی	transcript فارسی با قالب [MM:SS] name: text ، قابل ویرایش پیش از تحلیل.
محدودیت	حداکثر ۲۰ مگابایت (MAX_AUDIO_BYTES).
پس از رونویسی	همان مسیر POST /api/meetings — تحلیل، Jira، RAG، Chroma.

تفاوت با STT حرفه‌ای: diarization و جریان زنده بخشی از MVP نیستند؛ برای دمو و صوت ضبط‌شده، رونویسی تک‌مرحله‌ای Gemini کافی است.

۵. SQLite – ذخیره جلسات

مورد	مقدار
فایل	<code>data/meetings.db</code>
جدول	<code>meetings</code> – transcript, summary, key_points, decisions, tasks (JSON)
مهاجرت	فایل‌های قدیمی <code>data/meetings/*.json</code> یک‌بار در اولین اجرا import می‌شوند.

SQLite به MVP پایداری بادوام و بدون عملیات می‌دهد. schema عمداً ساده است تا مهاجرت Sprint 5 به PostgreSQL (با SQLAlchemy + Alembic) سراسر باشد.

۶. ChromaDB – Vector Store

چرا Chroma؟ نسخه‌ی قبلی از یک ذخیره‌ی JSON دستی با حلقه‌ی cosine در Python استفاده می‌کرد – برای دمو کند و برای نگهداری سخت. ChromaDB embedded است: بدون سرور جدا، ذخیره در `data/chroma/`، و جستجوی سریع HNSW.

مورد	مقدار
کتابخانه	<code>chromadb</code>
Collection	<code>meeting_chunks</code>
فیلتر	<code>metadata.meeting_id</code>
Embedding	<code>gemini-embedding-001</code> (از Google API؛ Chroma فقط ذخیره/جستجو)

۷. عامل‌های Pydantic AI

analysis_agent ۷.۱

```
from pydantic import BaseModel
from pydantic_ai import Agent

class MeetingAnalysis(BaseModel):
    title: str
    summary: str
    key_points: list[str]
    decisions: list[str]
    tasks: list[MeetingTask]

analysis_agent = Agent(
    'google:gemini-3.5-flash',
    output_type=MeetingAnalysis,
    system_prompt='جلسه فارسی. فقط بر اساس متن transcript تحلیل',
)
```

۷.۲ RAG – رفتار هوشمند

- **سلام / hello:** بدون جستجو در transcript – پاسخ کوتاهی که به پرسش واقعی درباره‌ی جلسه دعوت می‌کند.
- **پرسش جلسه:** context ← Chroma top-4 داخلی برای LLM (transcript خام هرگز در AI نمایش داده نمی‌شود).
- **پاسخ:** فارسی طبیعی با نسبت‌دادن («علی گفت...») و در صورت نیاز یک footnote کوتاه.

```
# small talk تشخیص
is_small_talk("hello") -> chitchat_agent, sources=[]
```

```
# پرسش واقعی
chunks = chroma.query(meeting_id, question)
rag_agent.run(context + question) -> RagAnswer
```

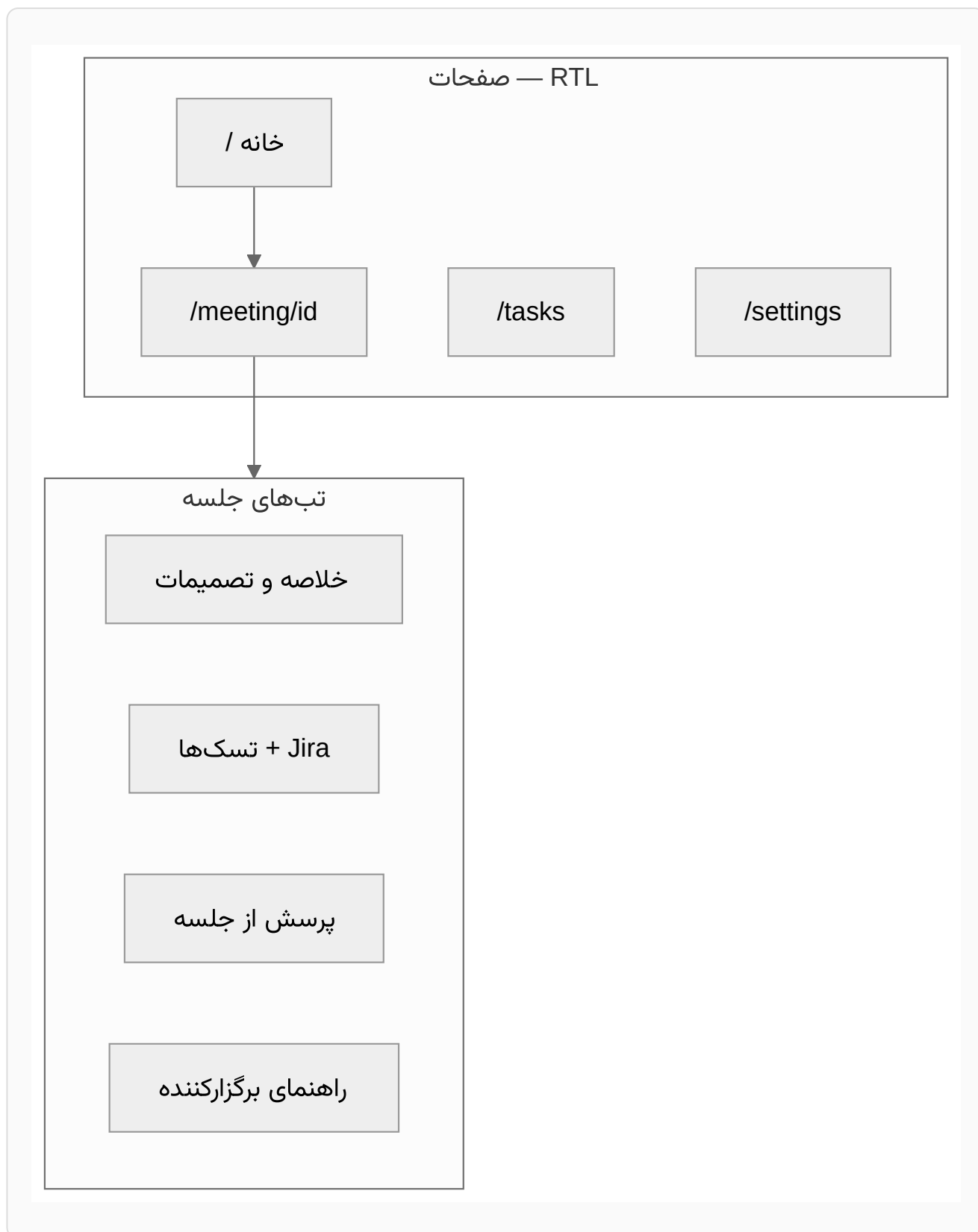
facilitation_agent ۷.۳

پس از ingest، برگزارکننده می‌تواند پیشنهادهای بهبود جلسه‌ی بعد را ببیند — با لحن مربی‌گری، نه سرزنش.

- **ورودی:** MeetingAnalysis، transcript، آمار مشارکت گویندگان، نوع جلسه.
- **خروجی:** FacilitationReport — نکات مثبت، بهبودها، دستور کار جلسه‌ی بعد، پیشنهاد timebox، خلاصه‌ی مربی‌گری، و امتیاز اختیاری ۱ تا ۵ برگزارکننده.
- **API:** GET /api/meetings/{id}/facilitation
- **UI:** تب «راهنمای جلسه» در صفحه‌ی جلسه (lazy load).

ایجاد Jira از یک route مستقیم API است. issue به انگلیسی (title_en, context_en) نوشته می‌شوند تا نمایش چپ‌به‌راست در Jira تمیز باشد، در حالی که ال فارسی می‌ماند.

۸. رابط کاربری (Astro SSR)



قواعد RTL/LTR:

- `html lang="fa" dir="rtl"` – متن فارسی راست‌چین.
- کد، متغیر محیطی، URL و نام API در `dir="ltr"` قرار می‌گیرند.
- نمودارهای Mermaid در یک wrapper با جهت LTR رندر می‌شوند.

صفحه‌ی `/meeting/{id}` داده را روی سرور **Astro** از FastAPI می‌گیرد (نه فقط با JS سمت کلاینت)، که مشکل «گیرکردن روی در حال بارگذاری» را رفع و first paint را بهتر می‌کند.

خانه – آرشیو جلسات، جستجو/فیلتر، جلسات مصنوعی، فرم ingest

جلسه ۰ خلاصه – نکات کلیدی، تصمیمات، آمار مشارکت

جلسه ۰ تسک‌ها – action item ها، preview و ایجاد issue در Jira

جلسه ۰ پرسش – RAG با پاسخ طبیعی و منابع

تسک‌ها – داشبورد cross-meeting با فیلتر «فقط باز»

تنظیمات – وضعیت API، نگاشت گوینده – Jira، راهنمای env.

اسکرین‌شات‌ها با (Playwright) `./scripts/capture-ui-screenshots.sh` تولید می‌شوند – backend و frontend باید در حال اجرا باشند.

۹. Endpoint های API

توضیح	Path	Method
سلامت سرویس – وضعیت Google/Jira/DB	/api/health	GET
لیست جلسات	/api/meetings	GET
دریافت یک جلسه	/api/meetings/{id}	GET
حذف جلسه	/api/meetings/{id}	DELETE

cross-meeting action item های	/api/tasks	GET
لیست جلسات مصنوعی همراه	/api/meetings/synthetic	GET
ingest یک جلسه ی مصنوعی	/api/meetings/synthetic/{file_id}	POST
رونویسی فایل صوتی با Gemini	/api/transcribe	POST
ingest + analysis_agent	/api/meetings	POST
خروجی خلاصه ی جلسه	/api/meetings/{id}/export	GET
آمار مشارکت گویندگان	/api/meetings/{id}/speakers	GET
راهنمای برگزارکننده – facilitation_agent	/api/meetings/{id}/facilitation	GET
RAG – rag_agent	/api/meetings/{id}/ask	POST
پیش نمایش issue های Jira	/api/meetings/{id}/jira/preview	POST
ایجاد issue در Jira	/api/meetings/{id}/jira/create	POST
نگاشت گوینده ← حساب Jira	/api/settings/assignee-map	GET / PUT / DELETE

۱۰. داده مصنوعی

موضوع	فایل
استنداپ هفتگی	meeting-01-scrum.txt
برنامه ریزی MVP	meeting-02-planning.txt
بازبینی sprint	meeting-03-review.txt

قالب خط: علی: متن گفتار [14:02] .

۱۱. تنظیمات محیط (env.)

```
GOOGLE_API_KEY=...
EMBEDDING_MODEL=gemini-embedding-001
GEMINI_MODEL=google:gemini-3.5-flash
MAX_AUDIO_BYTES=20971520
JIRA_SITE_URL=https://your-site.atlassian.net
JIRA_PROJECT_KEY=KAN
JIRA_EMAIL=your@email.com
JIRA_API_TOKEN=...
PUBLIC_BACKEND_URL=http://127.0.0.1:8000
BACKEND_PORT=8000
```

۱۲. ساختار پروژه

```
project/
├── frontend/          # Astro SSR – رابط فارسی RTL
├── backend/
│   ├── main.py        # FastAPI + routeها
│   ├── agents/        # analysis_agent, rag_agent, facilitation
│   ├── services/      # sqlite, chroma, jira, embed, audio_transcribe,
│   │                   # gemini_errors, stats, ingest, facilitation
│   ├── models/        # schemaهای Pydantic
│   └── tests/         # live + تست واحد
├── data/synthetic/    # جلسات نمونه
├── data/meetings.db   # خلاصه و تسکها – SQLite
├── data/chroma/       # پایداری ChromaDB
├── docs/              # CHANGELOG + نمودار + PDF + HTML اسناد
└── scripts/           # run-dev, run-backend, run-frontend, tests, export-docs
```

۱۳. اجرای سامانه

کل پشته با یک اسکریپت اجرا می‌شود (پورت‌ها را آزاد می‌کند، هر دو سرور را بالا می‌آورد، و هنگام خروج پاک‌سازی می‌کند):

```
chmod +x scripts/run-dev.sh
./scripts/run-dev.sh
# باز کنید http://localhost:4321
```

یا دو سرویس را جدا اجرا کنید:

```
./scripts/run-backend.sh # FastAPI 8000: روی
./scripts/run-frontend.sh # Astro 4321: روی
```

۱۴. تست‌ها

نوع	دستور	تعداد
واحد (mock)	./scripts/run-tests.sh	۱۵۰ موفق (۱۳ مورد بدون اعتبارنامه‌ی اختیاری/live رد می‌شوند)
live API و یکپارچگی	./scripts/run-live-tests.sh	۱۳ (Gemini/Jira واقعی)
همه	./scripts/run-all-tests.sh	واحد + live

تست‌های واحد، LLM و سرویس‌های بیرونی را mock می‌کنند تا سریع و آفلاین اجرا شوند. مجموعه‌ی live، route‌های واقعی FastAPI را در برابر اعتبارنامه‌ی واقعی Google/Jira اجرا می‌کند، با یک جریان end-to-end تکی Gemini در هر session تا از مشکلات event-loop در Pydantic AI جلوگیری شود.

۱۵. مدیریت خطا

خطا	رفتار
-----	-------

یک بار retry + پیام فارسی	Gemini در timeout
برچسب [ناشناس]	transcript بدون speaker
راهنمای بررسی .env	Jira 401
MAX_AUDIO_BYTES یا فرمت – HTTP 400 (فارسی)	فایل صوتی نامعتبر / بزرگ
پیام فارسی راهنمای billing برای رونویسی و تحلیل	Gemini billing (403 dunning)
«در این جلسه یافت نشد»	RAG بدون match

یک سرویس اختصاصی `gemini_errors`، استثنای ارائه دهنده (`ModelHTTPError`) و خطاهای `httpx` را به `HTTPException` های فارسی و کاربرپسند نگاشت می کند، تا یک `hold` در `billing` یا کلید نامعتبر هرگز به صورت یک `stack trace` خام ظاهر نشود.

اسپرینت های بعدی: به سند ۳ مراجعه کنید – RAG چندجلسه + آرشیو (Sprint 2) `full-text`، نگاشت `OAuth` + `assignee` در `auth/workspace`، (Sprint 3) `Jira` + متادیتای تقویم (Sprint 4)، `queue` + `PostgreSQL` + استقرار (Sprint 5)، و قابلیت های چندعامله (Sprint 6) `STT` جریانی زنده عمداً در نقشه ی راه نیست – رونویسی Gemini نیاز MVP را پوشش می دهد.